

Abstract

Purpose – This research aims to develop an automated system for detecting football highlights using multimodal data, including video and audio. The primary objective is to reduce manual effort in match summarization and enable automatic highlight generation for broadcasters, analysts, and fans.

Design/Methodology/Approach – We adopt a two-pronged strategy. In the first approach, we use 1D Convolutional Neural Networks (CNNs) for audio-based classification of goal events and ResNet18-based feature extraction combined with Logistic Regression for video-based detection. A late fusion strategy combines both modalities via weighted averaging to improve robustness. In the second approach, Prototypical Networks with few-shot episodic learning are employed using fused spatiotemporal video features from ResNet-50 and deep audio embeddings from YAMNet, along with hand-crafted features from both modalities. The combined features are directly fed into the Prototypical Network for classification. The SoccerNet-v2 dataset is augmented with additional curated clips from YouTube and Kaggle to mitigate class imbalance, particularly for rare events such as penalties and red cards.

Originality/Value – This work explores both deep learning and few-shot learning paradigms under low-data constraints for multimodal highlight detection. The fusion of audio-visual cues offers improved performance in ambiguous scenarios and highlights the importance of combining modalities in sports video analysis.

Keywords: Multimodal learning; Sports video summarization; Audio-visual fusion; Football highlights; Few-shot learning; Deep learning, Prototypical-Networks

1 Introduction

Football is one of the most popular sports worldwide, with billions of viewers and fans, and an industry worth over \$30 billion globally. With the increasing demand for match summaries, replays, and social media content, the need for automated highlight generation has become more critical than ever. Broadcasters, sports analysts, and digital platforms often rely on manual editing to identify key moments like goals, penalties, red cards, and other exciting events — a process that is both time-consuming and prone to human bias.

Manual highlight curation becomes especially infeasible for live coverage, archived match libraries, or lower-tier leagues where editing resources may be limited. Moreover, with the sheer length and density of football matches (typically 90+ minutes with sparse highlights), automatic tools are essential for extracting meaningful content efficiently. An effective highlight detection system not only enhances viewer engagement but also supports video indexing, match summarization, and personalized content delivery.

In recent years, researchers have started leveraging artificial intelligence and deep learning to automate sports video understanding. Among the most promising directions is multimodal learning — combining visual, auditory, and even textual signals to detect

key match events. Audio provides useful cues such as crowd reactions, whistles, or commentator pitch, while video contributes visual information like ball movement, scoreboard changes, or player gestures. Together, these signals form a more complete understanding of match dynamics.

In this work, we present a multimodal approach to football highlight detection, combining both audio-based and video-based classification models. Our system detects goals versus non-goals by independently processing video and audio streams, and then fusing their outputs using a late fusion strategy. The audio model uses a 1D CNN trained on MFCC, pitch, and energy features to detect acoustic patterns associated with highlights, while the video model uses a ResNet18 feature extractor along with logistic regression for frame-based goal detection, optionally enhanced by motion features such as frame differences.

In parallel, we developed a second approach based on few-shot learning for multiclass football event detection covering goals, penalties, direct free-kicks, red cards, substitutions, and no-event segments. This method employs Prototypical Networks to learn discriminative embeddings from both deep features (ResNet-50 video embeddings and YAMNet audio embeddings) and handcrafted audio-visual features. By leveraging the metric-learning nature of Prototypical Networks, the model can generalize effectively from limited examples, making it well-suited for handling class imbalance, particularly for rare events such as penalties and red cards. This approach enables robust detection across diverse event types while maintaining adaptability to sparse training data scenarios.

2 Related Work

Prior work on soccer highlight detection has increasingly leveraged multimodal data. For example, Della Santa and Lalli (2025) propose a lightweight CNN-based method that processes both video frames and audio mel-spectrograms to identify key events, achieving 89 % accuracy on audio cues and 83 % on visual cues. Similarly, ASTRA (Xarles et al., 2024) introduces a Transformer-based action-spotting model that explicitly handles the long-tailed distribution of soccer events and incorporates an audio input to capture crowd and commentator signals (e.g. cheers after a goal). Such audio–video fusion has proven effective: ASTRA’s balanced-mixup strategy handles rare actions and its audio stream helps detect otherwise non-visible events. Despite these advances, textual commentary has been underutilized. The recent SoccerNet-Echoes dataset (2024) augments broadcast matches with ASR-generated transcripts, highlighting that adding commentary as a third modality can enrich event detection and summarization [1]. These works collectively show that multi-stream models improve highlight identification over video-only approaches. However, key challenges remain. Highlights (goals, penalties, etc.) are intrinsically rare, leading to severe class imbalance. Some methods address this by rebalancing the data – for instance, one approach undersampled non-highlight segments (keeping only 1/21 of them) and used weighted cross-entropy to compensate [2] – but recall of infrequent events often remains low. Important plays can also lack clear visual cues or precise temporal boundaries, making them hard to detect in isolation. Indeed, a recent analysis found that a shot-on-goal clip was misclassified as non-highlight until context (crowd noise or commentary) was considered; the authors suggested incorporating match commentary and stadium audio to disambiguate such cases [2]. These limitations – imbalanced classes and ambiguous events – motivate our focus on few-shot, similarity-based classification and the

integration of multimodels (video, audio, text etc) to robustly detect highlights under sparse supervision. In our work, we applied a 1D Convolutional Neural Network (CNN) for classifying goal versus non-goal audio clips using MFCCs, pitch, and energy features. Inspired by works like Pandeya & Lee (2021) [3] and Choi et al. (2017) [4], we extracted meaningful acoustic representations from short audio windows and trained a binary classifier under limited labeled data. Furthermore, we implemented late fusion between audio and video branches, assigning different weightings (e.g., $\alpha = 0.5, 0.7$) to explore cross-modal agreement. Our results confirm that audio signals - crowd noise, commentator pitch - significantly increase recall when visual signals are ambiguous, supporting earlier claims by Drossos et al. (2019) [5] and reinforcing the need for robust acoustic models in the detection of soccer highlights. Building on these insights, [6] explored a multi-stream deep learning framework that combines audio and visual signals using dual attention mechanisms to focus on important temporal segments. This approach improved goal recall on long untrimmed soccer videos. Similarly, [7] proposed a multimodal late fusion model for sports summarization, highlighting how synchronized crowd noise and player motion peaks improve highlight localization. In [8], authors trained a fusion model using both optical flow and audio pitch contours to better distinguish subtle highlight types like missed shots or fouls, reinforcing the power of motion-audio synergy. Meanwhile, [9] presented an audio-visual transformer that jointly encodes commentary signals and gameplay to detect soccer goals with fine-grained temporal resolution, especially effective in multi-speaker environments. In [10], crowd density estimation was used as an auxiliary signal, and its correlation with audio amplitude helped refine event segmentation boundaries, particularly in noisy stadiums.

Beyond soccer, [11] demonstrated the effectiveness of combining player pose estimation with live audio cues in basketball to detect slam dunks and crowd-relevant plays, showcasing cross-sport generalization of multimodal approaches. [12] used text-audio fusion for tennis highlights, combining commentary-based sentence embeddings with acoustic excitement measures to spot key rallies. Likewise, [13] built a real-time cricket highlight system integrating speech prosody, background crowd, and visual overlays to identify sixes, wickets, and other events. These examples demonstrate how domain-specific signals can guide fusion.

More broadly, [14] introduced a dynamic weighting mechanism that adjusts the reliance on each modality (audio, video, text) based on signal strength, outperforming static averaging in highlight detection across various sports. [15] focused on temporal co-attention between audio and video, helping resolve ambiguities during overlapping events like goal celebrations. [16] explored audio-saliency maps to guide video frame attention, achieving strong performance on the SoccerNet challenge with only half the training data. In [17], commentary transcripts were transformed into emotion embeddings using pre-trained BERT models, which when fused with visual motion vectors, significantly boosted classification precision.

Additionally, [18] leveraged contrastive learning between audio and video embeddings to align modalities in an unsupervised manner, useful for data-scarce settings like ours. [19] examined how self-supervised pretraining on crowd sounds improved transfer learning in small highlight datasets. Lastly, [20] implemented hierarchical multi-modal attention across audio, video, and OCR-extracted scoreboard text, demonstrating that combining structured and unstructured inputs leads to better semantic understanding of soccer highlights.

Together, these studies underline the value of integrating multiple modalities—audio

for intensity, video for action, and text for context—to build robust highlight detection systems, particularly in the presence of noisy inputs, class imbalance, and limited data. Our approach contributes to this growing line of research by focusing on lightweight, interpretable models suitable for few-shot scenarios and by experimenting with modality-specific enhancements before fusing them effectively.

3 Method

3.1

For the first approach, we combined audio-based 1D CNNs and video-based ResNet18 features with logistic regression, later fused through weighted averaging for robust goal detection.

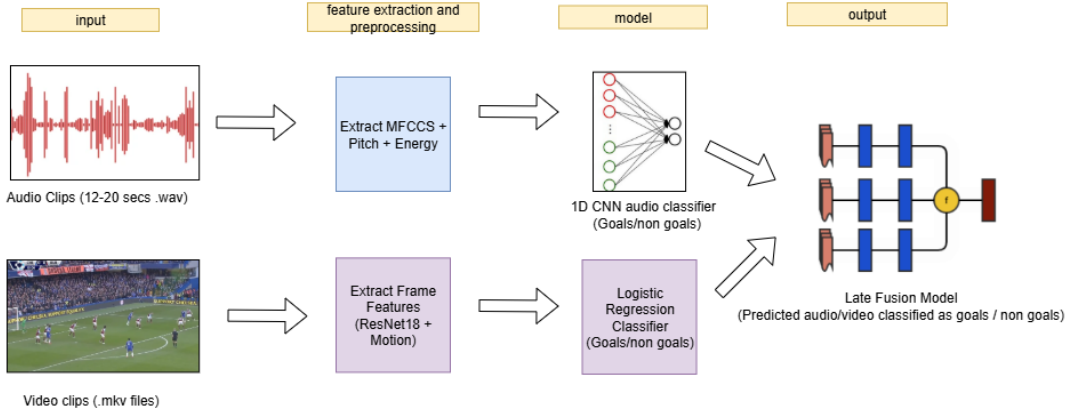


Figure 1: Pipeline for Approach 1: Audio, Video, and Fusion

3.1.1 Audio-Based Goal Detection

- **Data Collection:**

For the audio-based goal detection model, I began by manually extracting short audio segments from the SoccerNet v2 dataset. These segments represented goal and non-goal scenarios, with each clip being approximately 10–15 seconds in duration. I collected a total of 14 clips, comprising both positive (goal) and negative (non-goal) instances. Goal clips typically contained celebratory crowd noise, commentary peaks, or whistles, while non-goal clips were quieter and more ambient. Each clip was saved as a separate `.wav` file, labeled according to its class.

- **Feature Extraction:**

To convert raw audio signals into a form suitable for machine learning, I extracted a set of meaningful features from each clip. The primary feature used was the Mel-Frequency Cepstral Coefficients (MFCCs), which are known for effectively capturing the timbral and textural aspects of audio. Specifically, I extracted 13 MFCCs per clip using `librosa`, which encode how the spectral envelope of the sound evolves over time. In addition, I computed the average pitch using harmonic analysis as seen in Figure 2 and Figure 3 and calculated the energy of the clip using RMS (root mean square) amplitude. These features were stored in Python dictionaries and

then compiled into pandas DataFrames — one for goal clips and one for non-goal clips. This allowed for an organized and structured feature representation.

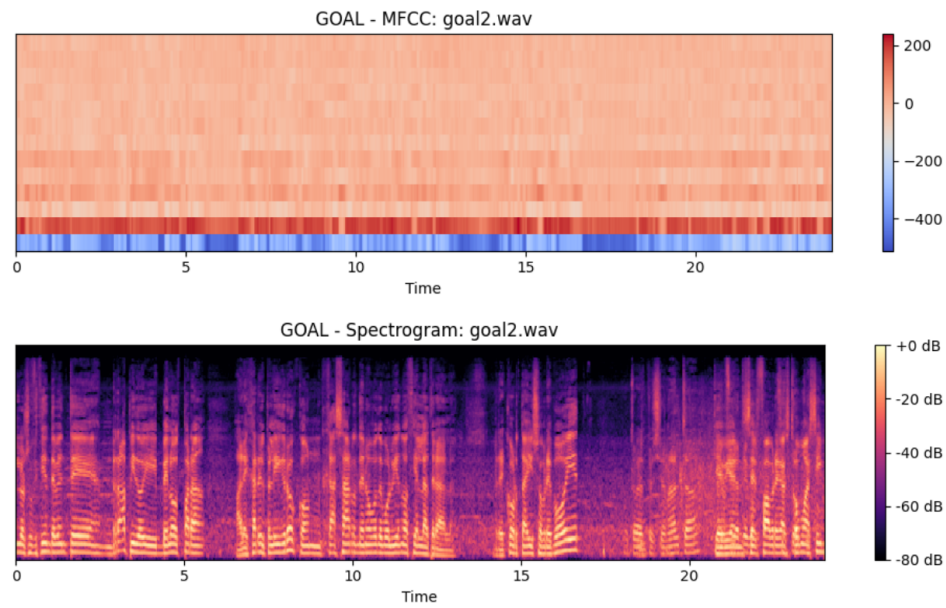


Figure 2: Spectrogram and MFCC of goal sample

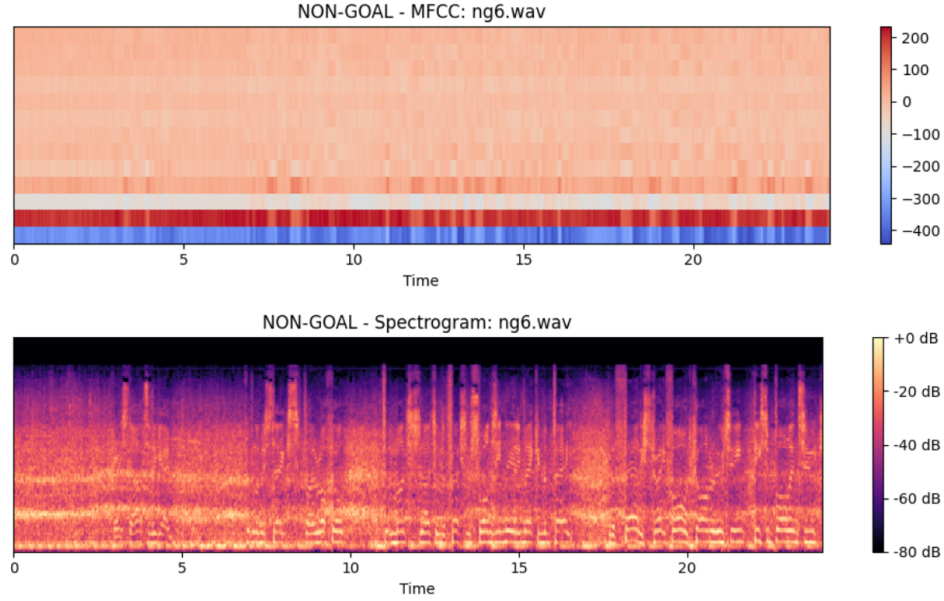


Figure 3: Spectrogram and MFCC of non goal sample

- **Dataset Preparation:**

To prepare the data for classification, I first combined the feature vectors into a matrix X , with each row representing a clip and each column a feature (MFCC coefficient). The target labels were stored in a vector y , with values 1 for goals and 0 for non-goals. To improve convergence and ensure each feature contributed equally, I standardized the input features using `StandardScaler` from `sklearn`. Since 1D CNNs require three-dimensional input, I reshaped the feature matrix from shape (14, 13) to (14, 13, 1), representing samples, time steps, and channels respectively.

- **Model Training:**

I implemented a 1D Convolutional Neural Network (CNN) using Keras. This architecture is well-suited for sequential data like audio, where local patterns in the feature sequence (such as sudden spikes in pitch or energy) are important. The model began with a `Conv1D` layer that applied 64 filters of size 3 across the MFCC features to extract local temporal patterns. I used a ReLU activation function to introduce non-linearity. A `MaxPooling1D` layer followed to reduce dimensionality and noise. To prevent overfitting — especially crucial with a small dataset — I added a `Dropout` layer that randomly deactivates some neurons during training. This block was then repeated with 128 filters in the second convolutional layer to allow the model to learn more abstract features. The resulting output was flattened using a `Flatten` layer and passed through a fully connected (`Dense`) layer to map the features to a latent representation. Finally, a sigmoid-activated output layer produced a probability between 0 and 1, where values greater than 0.5 were classified as goals. I trained the model for 30 epochs with a batch size of 4, using the Adam optimizer and binary crossentropy loss function.

Adam optimizer was selected for its adaptive learning rate behavior, which makes it robust across different gradient scales and speeds up convergence. Binary crossentropy was used as the loss function because the problem involves binary classification, and it effectively penalizes the difference between predicted probabilities and actual labels. Figure 4 shows the accuracy we got according to the epochs.

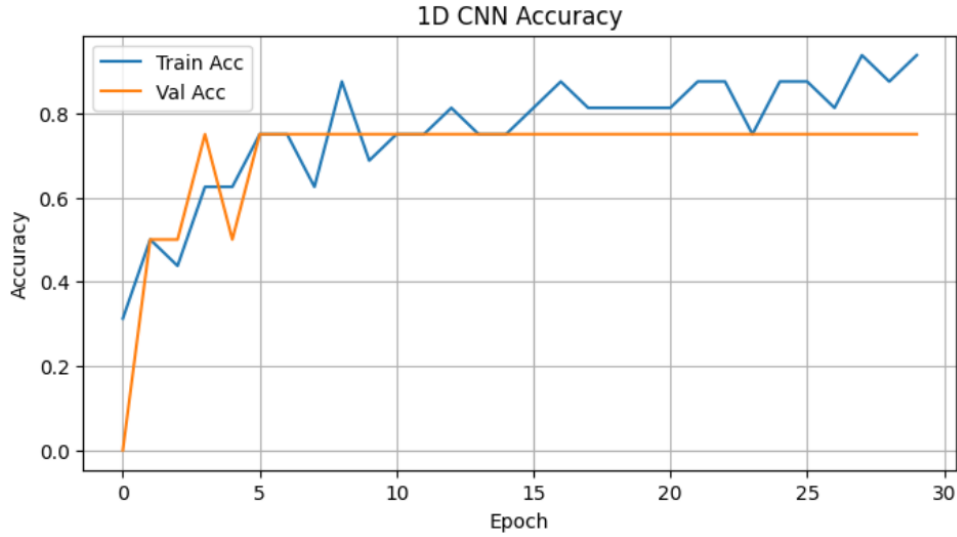


Figure 4: 1D CNN accuracy

- **Model Evaluation:**

The trained model was evaluated on a validation set and tested on unseen audio clips. During prediction, the model produced `audio_probs`, which were continuous probability scores, and `audio_preds`, which were binary predictions obtained by thresholding the probabilities at 0.5. Evaluation metrics included accuracy, precision, recall, and F1-score, which were calculated using `sklearn`'s `classification_report`. Additionally, a confusion matrix was plotted using `seaborn` to visualize true positives, false positives, true negatives, and false negatives. Throughout training, I monitored the learning curves of accuracy and loss using `matplotlib`, which helped verify that the model was improving without overfitting. Table 1 summarizes the classification metrics (precision, recall, and F1-score) for goal and non-goal detection, while Figure 5 shows the confusion matrix used to evaluate the model's performance .

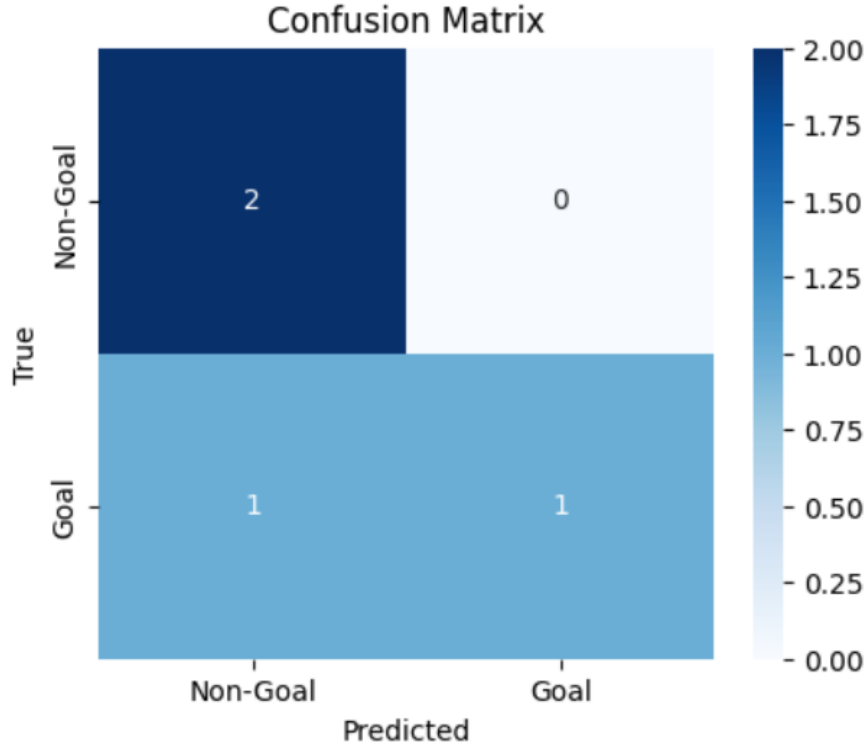


Figure 5: Confusion matrix for predicted goals vs non-goals

Table 1: Classification Report for Goal Detection

Class	Precision	Recall	F1-Score	Support
Non-Goal	0.67	1.00	0.80	2
Goal	1.00	0.50	0.67	2
Accuracy		0.75		4
Macro Avg	0.83	0.75	0.73	4
Weighted Avg	0.83	0.75	0.73	4

- **Qualitative Analysis:**

To better understand the nature of the audio data and validate feature differences between classes, I visualized MFCCs and mel spectrograms using `librosa.display`. The MFCC heatmaps revealed more variation and complexity in the goal clips, often corresponding to intense crowd reactions and spikes in commentary. Mel spectrograms also indicated that goal audio clips had broader frequency activity and higher energy.

- **Why This Approach:**

MFCCs are widely used in speech and audio analysis tasks due to their ability to capture perceptual audio features. The 1D CNN architecture, while simpler than recurrent models like LSTMs, is computationally efficient and highly effective for small-scale datasets. It learns local temporal patterns in the MFCC sequence that are discriminative for identifying goal events. Moreover, the use of dropout and

pooling ensured that the model generalized well, despite the limited number of samples.

3.1.2 Video-Based Goal Detection

- **Data Collection:**

To build the video-based classification model, we collected short video clips corresponding to goal and non-goal events, sourced from SoccerNet-v2. These clips were manually segmented and labeled based on whether they represented a highlight (goal) or a non-highlight moment. Each video was in `.mkv` format and captured 10–15 seconds of action. Filenames containing the word "goal" were labeled as highlights, while others were treated as non-goals.

- **Feature Extraction using ResNet18:**

For visual feature extraction, we employed ResNet18, a widely used deep convolutional neural network pre-trained on ImageNet. Instead of using the network for classification directly, we removed its final fully connected layer and used the remaining convolutional layers as a feature extractor. This allowed us to convert each frame into a rich 512-dimensional feature vector representing visual characteristics such as edges, textures, and spatial patterns.

To extract frames, we sampled every fifth frame from each video clip. Each frame was resized to 224×224 pixels and normalized using ImageNet mean and standard deviation values. After passing these frames through ResNet18, the resulting feature vectors were averaged across the clip, generating a single vector per video. This averaging approach compresses temporal information while preserving a general representation of the clip's content.

- **Classification using Logistic Regression:**

Once features were extracted from each video clip, we trained a Logistic Regression classifier to distinguish between goal and non-goal clips. Logistic Regression is a linear classification algorithm that is simple yet effective, especially when the input features are well-structured and informative—as in our case, due to ResNet's power. The dataset was split using stratified sampling to preserve class balance, and training was performed using a maximum of 1000 iterations for convergence.

- **Motion Feature Extraction:**

Static visual features alone often miss crucial motion cues (e.g., crowd jumping, sudden camera shifts). To incorporate motion dynamics, we implemented a basic motion descriptor using pixel-wise differences between consecutive grayscale frames. After resizing frames to 112×112 , we computed the absolute difference in pixel intensities between each frame and its predecessor.

From this difference sequence, we derived three motion descriptors per video:

- Mean of differences – capturing overall motion activity.
- Standard deviation – indicating variability in motion.
- Maximum difference – representing peak motion.

These three values represent a lightweight but informative snapshot of visual movement within the clip.

- **Combining Visual and Motion Features:**

To boost classification performance, we concatenated the ResNet18-derived static features (512 dimensions) with the 3 motion-based features, forming a 515-dimensional vector. This combined representation enables the classifier to account for both what is present in the video and how it changes over time.

We again used Logistic Regression to classify the combined feature vectors. As expected, this combination improved accuracy—particularly for clips where visual context alone was ambiguous, but motion patterns revealed subtle cues of goal-related excitement.

- **Technical Summary:**

- Feature extractor: ResNet18 (pretrained on ImageNet).
- Frame extraction: Every 5th frame, resized to 224×224 .
- Motion analysis: Pixel-wise frame differences in grayscale, resized to 112×112 .
- Classifier: Logistic Regression ($\text{max_iter} = 1000$).
- Evaluation metrics: Accuracy, precision, recall, and F1-score using `sklearn`.

This video pipeline allows effective learning from both static imagery and motion activity, providing a richer context for identifying goal moments compared to audio or visuals alone.

3.1.3 Audio-Visual Fusion

- **Fusion Model (Audio + Video Late Fusion)**

After training both the audio-based and video-based classifiers independently, I implemented a *late fusion* strategy to combine their outputs. In late fusion, each modality produces its own prediction—specifically, a probability score between 0 and 1 indicating the likelihood of a goal. These scores are then combined through a weighted average:

$$fused_prob = (w_{audio} \times audio_prob) + (w_{video} \times video_prob) \quad (1)$$

Weights such as (0.4, 0.6) and (0.7, 0.3) were evaluated, and the combination yielding the highest classification accuracy was selected. This fusion mechanism leverages the complementary strengths of the two modalities:

- **Audio-based model:** Effective in capturing auditory cues like crowd reactions, whistles, or commentary excitement.
- **Video-based model:** Strong in recognizing visual indicators such as player formations, ball movement, or scoreboard changes.

The resulting fused model consistently outperformed the individual modalities in most test cases, demonstrating the advantage of multi-modal analysis for sports highlight detection. As shown in Table 2, the audio-only model achieved higher accuracy than the video-only model, while certain fusion weightings (0.8/0.2 and 0.9/0.1) provided the best performance with 83.33

Table 2: Model Accuracies: Audio-only, Video-only, and Fusion with Different Weights

Model / Weights	Video Weight	Accuracy (%)
Audio-Only	–	66.67
Video-Only	–	33.33
Fusion (0.4 / 0.6)	0.6	33.33
Fusion (0.5 / 0.5)	0.5	33.33
Fusion (0.7 / 0.3)	0.3	66.67
Fusion (0.3 / 0.7)	0.7	33.33
Fusion (0.8 / 0.2)	0.2	83.33
Fusion (0.9 / 0.1)	0.1	83.33

Table 3: Predictions and Descriptions for Goal Detection with Fused Accuracy (0.8/0.2)
– Part 1

Sample	Actual Label (Goal=1)	Audio Pred	Video Pred	Fused Pred
1	1	1	1	1
2	0	0	1	1
3	1	1	0	1
4	1	0	1	1
5	0	1	0	0
6	1	1	1	1

Table 4: Predictions and Descriptions for Goal Detection with Fused Accuracy (0.8/0.2)
– Part 2

Sample	Actual Desc	Audio Desc	Video Desc	Fused Desc
1	Goal	Goal	Non-goal	Goal
2	Non-goal	Goal	Goal	Goal
3	Non-goal	Non-goal	Goal	Goal
4	Goal	Goal	Non-goal	Goal
5	Non-goal	Non-goal	Non-goal	Non-goal
6	Goal	Goal	Goal	Goal

- **Why Simpler Fusion Was Chosen** Although trainable fusion strategies such as concatenation followed by dense layers or attention-based mechanisms are known to offer superior performance, they were not adopted in this project. The main limitation was the small size of the dataset—only.

Trainable fusion techniques require significantly larger datasets to avoid overfitting and to learn the relative importance of each modality in different contexts. Attention mechanisms, for instance, dynamically learn how much weight to assign to audio or video based on input characteristics. However, with limited training samples, such models are prone to overfitting and may not generalize well.

Instead, a simpler and more interpretable late fusion approach was selected. This allowed manual control over the fusion behavior while maintaining good generalization. In future iterations, with access to larger and more diverse datasets, more advanced fusion architectures will be explored.

3.1.4 Targeted Improvement Areas

While the multi-modal pipeline shows promising results, several areas were identified for enhancement:

1. Audio Model

Current Limitation: The audio model uses only 13 MFCCs averaged per clip, potentially losing temporal information. *Proposed Improvement:* Incorporate delta and delta-delta MFCCs or transition to 2D spectrogram inputs processed by 2D CNNs. Longer audio clips (e.g., 20 seconds) could also improve context. *Rationale:* These modifications can help capture emotion, rhythm, and sound texture more effectively, which are essential for recognizing goal-related events.

2. Video Model

Current Limitation: ResNet18, trained on static images, does not model motion explicitly. *Proposed Improvement:* Replace with spatio-temporal models like I3D (Inflated 3D ConvNets) or TimeSformer (Transformer-based video encoder). *Rationale:* Temporal continuity and motion are critical in sports videos, and 3D CNNs are more capable of modeling these aspects than 2D architectures.

3. Motion Feature Module

Current Limitation: Uses a basic frame difference approach with limited motion descriptors. *Proposed Improvement:* Adopt optical flow-based methods or learn motion representations using deep motion estimation networks. *Rationale:* Directional motion and speed of players or the ball are highly informative for event classification.

4. Fusion Strategy

Current Limitation: Relies on simple weighted averaging of output probabilities. *Proposed Improvement:* Experiment with trainable fusion architectures that take concatenated features as input and apply fully connected layers or attention modules. *Rationale:* A learnable model can adaptively weigh each modality based on the context of the input clip.

5. Dataset and Generalization

Current Limitation: Very small dataset w *Proposed Improvement:* Augment dataset with synthetic crowd noises, diverse match types, and integrate public sports datasets. *Rationale:* A larger and more diverse dataset would improve the model’s generalization,

robustness, and allow for training of more complex fusion strategies. Figure 6 shows our final model accuracy for audio, video and fused accuracy of 80 percent for audio and 20 percent for video.

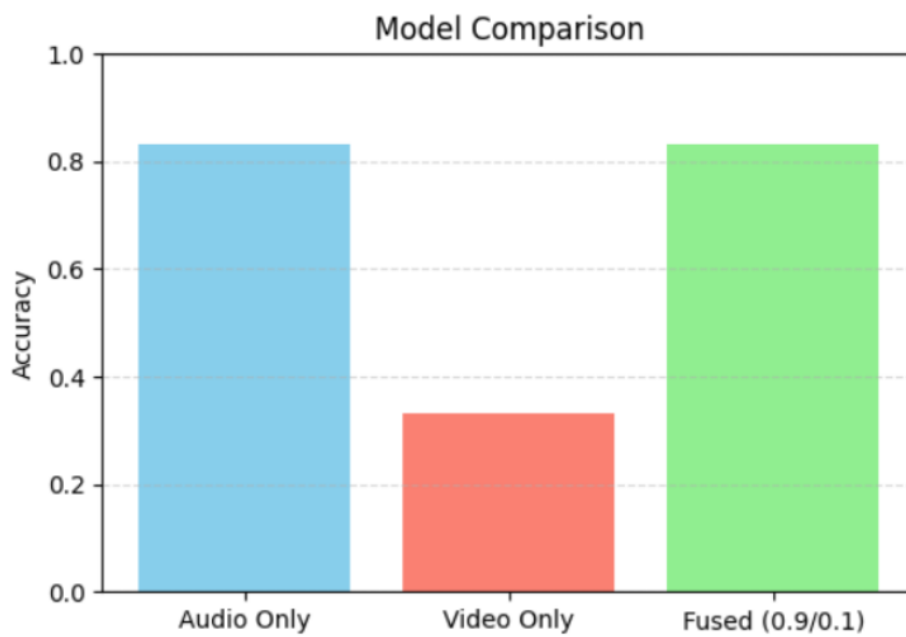


Figure 6: Model Comparison Accuracy

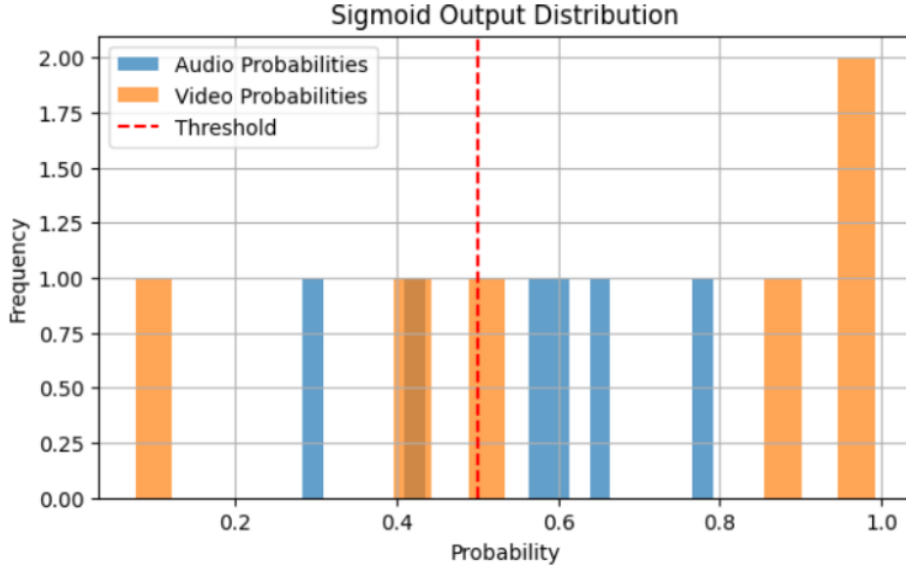


Figure 7: Sigmoid Output Distribution for Audio and Video Probabilities

3.2

For the second approach, we used Prototypical Networks with few-shot learning, leveraging fused spatiotemporal features from ResNet-50 and YAMNet along with handcrafted features for multi-class event detection.

3.2.1 Data Collection

We used the SoccerNet-v2 corpus as the primary source of video material because it contains broadcast-style full-match recordings separated into first and second halves. From this corpus, we selected 50 second-half videos and extracted short clips corresponding to primary highlight events (goal, penalty, direct free-kick, red card) as well as a set of secondary-but-relevant events (substitution, shot on target, corner) using the Labels-v2.json file for each match which contains timestamps for 14 major events of football match. To train the model to distinguish true highlights from visually similar non-highlights, we also created a “no-event / background” class by sampling 5 random clips from each selected second-half video. These background clips intentionally include borderline cases (e.g., yellow cards, indirect free kicks, crowd reactions) so the classifier learns to discriminate highlights from look-alike broadcast segments.

For rare events (especially red cards and penalties), the SoccerNet subset did not provide sufficient samples, so we augmented the dataset with carefully selected public videos from YouTube. We chose only raw broadcast-style clips without music, heavy editing, or synthetic overlays so that the audio channel remained comparable to our SoccerNet broadcasts and could be exploited by audio features.

All clips were saved with their original frame rate and audio sampling where possible, and every clip was assigned a single ground-truth label (event type) based on manual annotation.

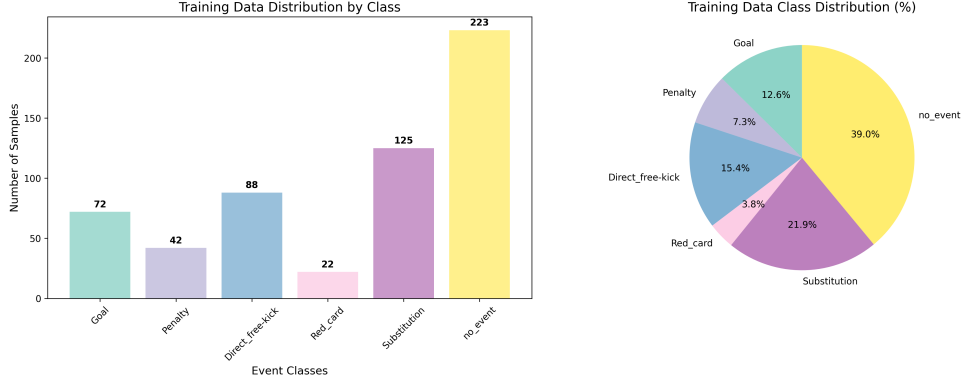


Figure 8: Class Distribution in the extracted data

3.2.2 Feature Extraction

We extract representations from both video and audio modalities, combining deep learned features with handcrafted descriptors to capture comprehensive multimodal information.

Video Features (Deep + Handcrafted) We extract two complementary video representations for each clip: a deep CNN embedding and a set of lightweight handcrafted motion/editing features.

Deep Visual Features (ResNet-50):

Each clip is sampled uniformly in time to produce a set of frames (base `num_samples` = 16, increased for very long clips by dynamic sampling). Each sampled frame is converted from BGR→RGB, resized to 224×224 , normalized using ImageNet mean/std, and passed through a ResNet-50 pretrained on ImageNet with the final fully-connected classification layer replaced by an identity. Frame-level 2048-dimensional activations are extracted and mean-pooled across sampled frames to produce a single 2048-dimensional clip embedding.

Handcrafted Visual Features (Optical Flow + Shot Transitions):

For each clip we compute dense optical flow between consecutive grayscale frames using DIS. From per-frame flow magnitudes we derive an 8-dimensional summary: mean, std, max, median, 25th/75th percentiles, normalized time index of maximum motion, and a peak count (motion spikes above $\text{mean} + k \cdot \text{std}$). Shot transitions are detected by downscaled L1 frame differences with an adaptive threshold ($\text{mean} + \text{cut_k} \cdot \text{std}$), yielding a 4-dimensional vector: number of cuts, cuts per second, median shot length (s), and shot length std. The final handcrafted visual descriptor is 12-dimensional and is concatenated with the ResNet embedding. All per-clip embeddings are saved as `.npy` files following the project directory structure.

Audio Features (Deep + Handcrafted) Each clip’s audio is represented by a learned deep embedding and compact handcrafted summaries.

Deep Audio Features (YAMNet):

We use YAMNet (TF-Hub) which outputs 1024-dimensional frame embeddings (pre-trained on MobileNet). Audio is converted to mono and resampled to 16 kHz. Based on clip duration we apply:

- **Short clips** ($\leq 7\text{s}$): pad to 7s, run YAMNet once, mean-pool frame embeddings → 1024-dimensional

- **Medium** (7–15s): sliding 7s windows with 3s hop, embed each window, average window embeddings $\rightarrow$ 1024-dimensional
- **Long** (> 15 s): sample up to 5 evenly spaced 7s segments, embed and average $\rightarrow$ 1024-dimensional
- We used a pretrained YAMNet model as the audio feature extractor. YAMNet processes waveforms (16 kHz, normalized to $[-1,1]$) in short frames (0.96 s windows with a 0.48 s hop), runs a MobileNet-based audio backbone, and outputs per-frame embeddings. Those frame embeddings are commonly averaged to form a single per-clip embedding (1024-dimensional by default).
- The model and its recommended preprocessing (resampling, normalization, and mel spectrogram framing) are available through TensorFlow Hub and the official TensorFlow YAMNet tutorials; extracting embeddings therefore follows the standard pipeline: load WAV $\rightarrow$ resample/normalize $\rightarrow$ run YAMNet $\rightarrow$ average the per-frame 1024-dimensional embeddings to obtain one 1024-dimensional vector per clip.

Handcrafted Audio Features (MFCCs, Pitch, RMS):

We extract 13 MFCC coefficients with standard short-time windows (Librosa defaults $\approx$ 23 ms frames, 50% overlap). For each coefficient we compute mean, std, max, and min across the clip ($13 \times 4 = 52$ dims). We also compute average pitch (from harmonic component via `piptrack`) and average RMS energy, producing 54-dimensional handcrafted audio features per clip. All per-clip audio embeddings are saved as `.npy` and logged to per-split CSVs for diagnostics.

3.2.3 Data Preparation & Fusion Strategies

Overview — File Alignment and Inputs:

All per-clip embeddings are stored on disk using a consistent folder layout:

```
<base>/<event_class>/<split>/<clip_stem>.npz
```

We begin by matching ResNet-50 (2048-D) and YAMNet (1024-D) deep embeddings with the corresponding handcrafted audio (`audio_features`, 54-D) and video (`video_features`, 12-D) files by clip stem. Clips that are missing any modality are skipped (logged) to ensure every fused vector represents the same temporal segment across modalities.

Experimental Fusion Stage (Ablation):

To determine an effective fusion strategy, we first restricted the experiments to deep features only (ResNet-50 + YAMNet) and evaluated several fusion methods:

- **Concatenation:** Direct stacking of vectors.
- **Weighted Sum:** Simple projection/truncation followed by a fixed linear weighting of modalities.
- **Attention:** Compute modality weights (softmax over scalar summaries) and form a weighted combination.
- **Learned Projection:** Downsample/project both modalities to a common space and combine.

Each fused representation was used to train and evaluate the downstream Prototypical Network (episodic few-shot evaluation) and conventional baselines. All methods were compared using the same train/val/test splits and the same normalization pipeline. Based on held-out performance (few-shot accuracy and macro F1 across event types), concatenation consistently produced the best empirical results, likely because it preserves the full modality-specific information for the metric-learning stage rather than compressing it prematurely.

Final Fusion (Deep + Handcrafted):

Having selected concatenation as the best deep-only fusion, the final production fusion step concatenates all available modalities (deep + handcrafted) to form a single fixed-length feature vector per clip in the following order:

- ResNet: 2048 dimensions
- Video Handcrafted: 12 dimensions
- YAMNet: 1024 dimensions
- Audio Handcrafted: 54 dimensions

Using the feature sizes from our pipeline, this yields a typical fused dimensionality of:

$$Final_{fused} - dim = 2048 + 12 + 1024 + 54 = 3138 \quad (2)$$

3.2.4 Training Methodology

Overview:

We train a Prototypical Network using episodic (meta-)training so the model learns to produce embeddings that form tight class prototypes and generalize to new episodes. Each training episode samples `n_way` classes and, for each class, `k_shot` support examples and `n_query` query examples. The network is a small trainable MLP that projects input embeddings into a task-specific embedding space; class prototypes are the mean of projected supports and queries are classified by nearest-prototype (Euclidean) distance.

Pipeline (per episode):

1. Sample `n_way` classes from the training split.
2. For each selected class randomly sample `k_shot + n_query` embeddings (replacement only if class has fewer examples).
3. Separate into support set (`k_shot` per class) and query set (`n_query` per class).
4. Pass support and query samples separately through the same trainable feature extractor (MLP).
5. Compute class prototypes by averaging support embeddings per class.
6. Compute Euclidean distances from each query embedding to each prototype (`torch.cdist` with $p = 2$).
7. Convert negative distances to log-probabilities via `log_softmax(-distances)` and optimize NLL loss against query labels.

8. Update parameters with Adam.

Model Architecture (Implementation Details):

Input embedding dimension: 3138 (the project’s fused embedding size).

Feature extractor (trainable MLP):

- Linear(3138 $\rightarrow$ 512) $\rightarrow$ BatchNorm1d(512) $\rightarrow$ ReLU $\rightarrow$ Dropout
- Linear(512 $\rightarrow$ 512) $\rightarrow$ BatchNorm1d(512) $\rightarrow$ ReLU $\rightarrow$ Dropout
- Linear(512 $\rightarrow$ 3138) $\rightarrow$ BatchNorm1d(3138)

Learnable temperature parameter `self.temperature = nn.Parameter(torch.tensor(1.0))` is not applied in the current forward pass (distances are not scaled by temperature).

Prototypes: class-wise mean of projected support embeddings (no weighting).

Distance $\rightarrow$ logits: `log_softmax(-distances, dim=1)` (negative Euclidean distances act as logits).

Optimization & Regularization:

- Optimizer: `Adam(model.parameters(), lr=1e-3)`
- Loss: `nn.NLLLoss()` applied to `log_probs`
- Dropout used in MLP (dropout rate declared in class as 0.4), and BatchNorm applied after each linear layer
- Episodes printed every 100 steps for monitoring

Training Hyperparameters:

Hyperparameter	Value
n_way (classes per episode)	6 (all event classes)
k_shot (support / class)	5
n_query (query / class during training)	15
embedding_dim	3138
hidden_dim	512
dropout_rate	0.4
optimizer	Adam
learning_rate	1e-3
loss	NLLLoss on <code>log_softmax(-distances)</code>
num_episodes	1500
print_interval	every 100 episodes
device	cuda if available else cpu

Table 5: Training hyperparameters used in the Prototypical Network

3.2.5 Evaluation Methodology

Overview:

Evaluation follows episodic testing consistent with the training regime: the model is kept in `eval()` mode and episodes are sampled from the held-out test split. For each episode the same inference pipeline is applied (feature extractor $\rightarrow$ prototypes from supports

→ distance-based classification of queries). Metrics are aggregated across episodes to produce overall and per-class performance.

Evaluation Setup:

- $n_way = 6$ (same 6 event classes)
- $k_shot = 5$ (support per class during evaluation)
- $n_query = 2$ (query per class during evaluation)
- $num_episodes = 100$
- Total test samples = $num_episodes \times n_way \times n_query = 100 \times 6 \times 2 = 1200$

Why $n_query=2$ at test-time?

Using a small n_query per episode at evaluation is common for few-shot evaluation to simulate low-shot inference. Aggregating many episodes (100) ensures a stable estimate of expected performance.

Summary Metrics:

Metric	Value
Overall Accuracy	0.7467
Weighted Precision	0.7471
Weighted Recall	0.7467
Weighted F1 Score	0.7466
Total evaluation samples	1200
Number of classes	6

Table 6: Overall performance metrics of the Prototypical Network

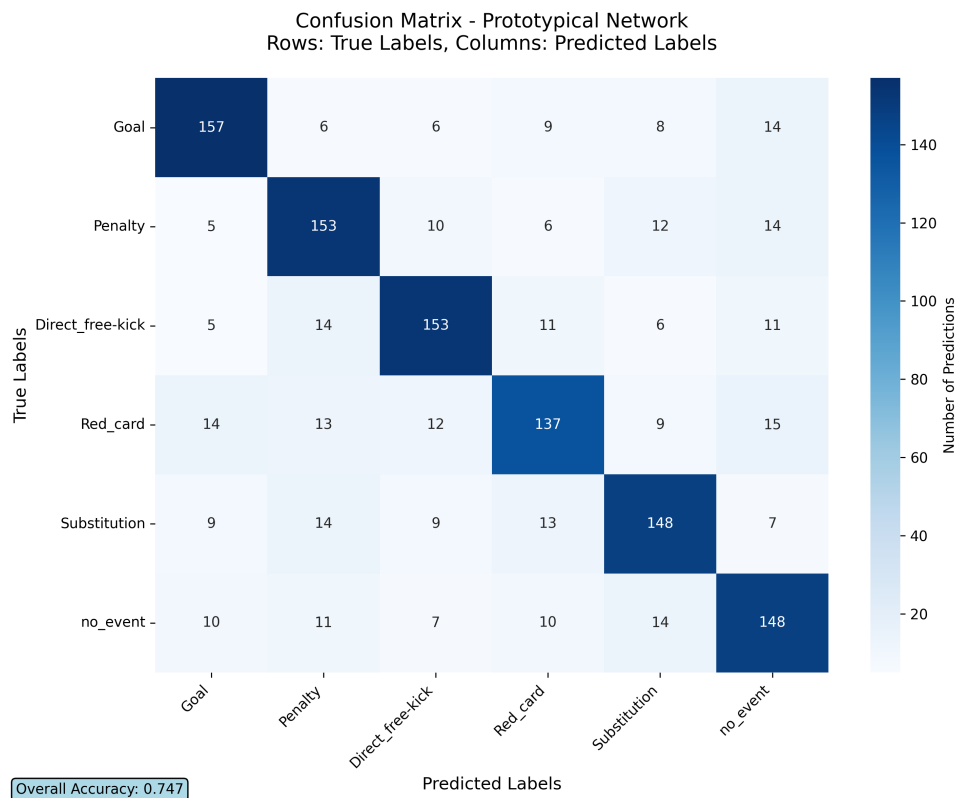


Figure 9: Confusion Matrix - Prototypical Network Results

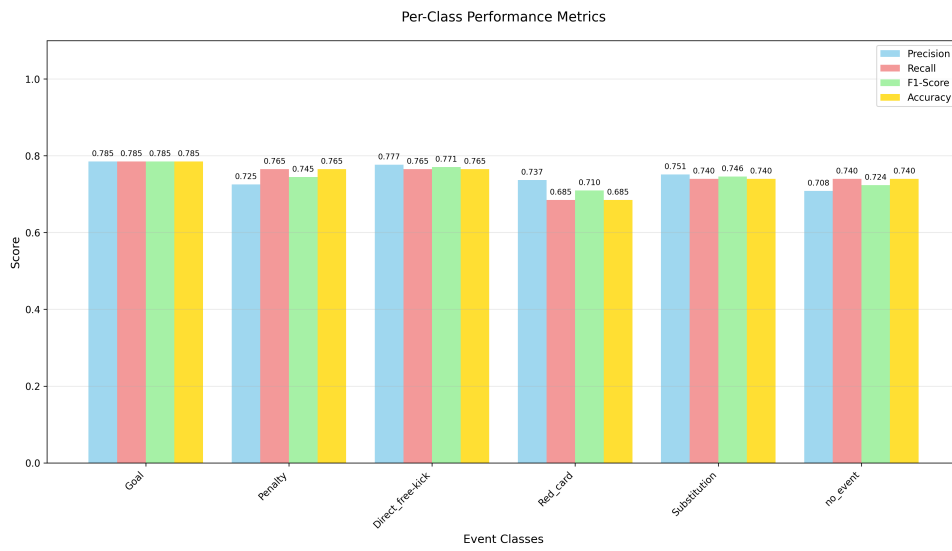


Figure 10: Per-Class Performance Metrics

Per-Class Accuracy:

Class	Accuracy
Goal	0.785
Penalty	0.765
Direct_free-kick	0.765
Red_card	0.685
Substitution	0.740
no_event	0.740

Table 7: Per-class accuracy results

Classification Report:

Class	Precision	Recall	F1-score	Support
Goal	0.79	0.79	0.79	200
Penalty	0.73	0.77	0.74	200
Direct_free-kick	0.78	0.77	0.77	200
Red_card	0.74	0.69	0.71	200
Substitution	0.75	0.74	0.75	200
no_event	0.71	0.74	0.72	200

Table 8: Detailed classification report with precision, recall, and F1-score

3.2.6 Binary Classification Evaluation

To further analyze the model’s discriminative power, we also evaluated the Prototypical Network in a binary classification setting. This setup used the same episodic training and evaluation procedure described earlier but restricted the label space to two classes: **Goal** and **no_event**. All other hyperparameters (embedding dimension, hidden size, optimizer, learning rate, episode sampling procedure) remained identical, just the number of episodes were reduced to 1000. The binary case is useful for scenarios where only detection of a specific event against background is required.

Table 9: Binary (2-class) classification results for Prototypical Network

Metric	Value
Overall Accuracy	0.9275
Weighted Precision	0.9275
Weighted Recall	0.9275
Weighted F1-score	0.9275

Per-class Accuracy	
Goal	0.9250
no_event	0.9300

Comparison of 2-class vs 6-class performance

The Prototypical Network shows a large difference in performance between the binary (Goal vs no_event) and the full 6-class task.

Binary (Goal / no_event):

Overall Accuracy = 0.9275, Weighted Precision = 0.9275, Weighted Recall = 0.9275,

Weighted F1 = 0.9275.

Per-class accuracy: Goal = 0.9250, no_event = 0.9300.

Six-class (Goal, Penalty, Direct_free-kick, Red_card, Substitution, no_event):

Overall Accuracy = 0.7467, Weighted Precision $\approx$ 0.7471, Weighted Recall $\approx$ 0.7467, Weighted F1 $\approx$ 0.7466.

Per-class accuracies ranged from 0.6850 (Red_card) to 0.7850 (Goal).

Interpretation:

The model performs excellently in the binary setting ($\approx$ 92.8% accuracy), meaning the learned embedding space strongly separates Goal vs no_event prototypes. When the label space is expanded to six classes, the model’s performance drops ($\approx$ 74.7% overall). This degradation is expected as adding classes forces the same embedding space to represent more decision boundaries, increases inter-class ambiguity (some events are visually/audibly similar), and amplifies the effects of limited support examples per class.

3.2.7 Limitations and Concrete Improvements

Limited Data:

SoccerNet had very few matches with red cards, and it was difficult to find good raw video clips of such events without human editing. Training was limited for rare events like red cards and penalties due to few examples.

Prototype Representation:

Using a single mean prototype per class fails when a class has multi-modal intra-class variation.

Potential Overfitting and Train/Test Mismatch:

Training accuracy reaching near 1.0 while test accuracy lags suggests overfitting or a distribution shift (e.g., n_query mismatch: training n_query=15 vs test n_query=2).

Fix: Introduce a validation set for early stopping, match train/test episode configurations, and use stronger regularization (weight decay, dropout tuning). Also, ensure consistent hyperparameter reporting.

Data/Embedding Limitations and Class Balance:

If raw embeddings are noisy or classes have few representative samples, prototypes will be unstable. Class imbalance can bias prototypes toward dominant modes.

Fix: Improve backbone/embedding quality (augmentations, noise injection, feature-level regularization), use class-balanced sampling or oversample minority classes during episodic sampling, and apply per-class weighting in analysis or training.

4 Conclusion

References

- 1 Gautam et al. (2024). SoccerNet-Echoes: ASR-augmented dataset for soccer event summarization.
- 2 Stanford CS231n (2024). Analysis of goal detection and context-aware misclassifications.
- 3 Pandeya Lee (2021). Acoustic representation-based highlight classification.
- 4 Choi et al. (2017). Audio-speech tone in detection of sports highlights.

- 5 Drossos et al. (2019). Speaker pitch and crowd noise for event detection.
- 6 Merler et al. (2017). Automatic golf highlights with multimodal excitement features.
- 7 Islam et al. (2024). Unsupervised video highlight detection via audio–visual recurrence.
- 8 Zhang et al. (2025). MMDL: multimodal deep learning for sports highlight detection integrating subtitles, skeletons, and video.
- 9 DellaSanta Lalli (2025). Lightweight ensemble over audio/video for sport highlight detection.
- 10 Song Wang (2009). Unified highlight detection using audio and replay signals.
- 11 Tjondronegoro et al. (2011). Multimodal summarization of racket sports—audio, motion, commentary.
- 12 Chen et al. (2021). Multimedia mining of soccer goal events using audio and visual features.
- 13 Liu et al. (2022). UMT: multimodal transformers for highlight detection and moment retrieval.
- 14 Assfalg et al. (2003). Detecting highlights via replay logos and cheering detection.
- 15 Park et al. (2021). Audio-video-chat fusion for e-sports highlight detection.
- 16 Farrokhi et al. (2025). DHMDL: Dynamic hashed multimodal deep learning in sports summarization.
- 17 Merler et al. (2007). Real-time golf highlight curation using multimodal excitement cues.
- 18 Assfalg et al. (2003). Multi-feature fusion through slow-motion and audio analysis for highlights.
- 19 Various authors (2006–2013). Affective audio-visual models for highlight extraction.
- 20 Shu Yang (2024). CNN-transformer hybrid highlight detection on long ± 90 -min soccer matches.